4章

区切られた文脈どうしの連係

区切られた文脈は、同じ言葉の一貫性を守るだけなく、モデルを作る枠組みを提供します。モデルの構築には、目的の特定、つまり境界が必要です。境界は同じ言葉が通用する範囲を分割します。ある区切られた文脈の同じ言葉は、特定の課題を解決するために、事業活動をモデル化します。同じ事業要素（ヒト、モノ、コト）を対象に、もう一つ別の区切られた文脈を設定すれば、異なる問題を解決するための別のモデルを構築できます。

さらに言えば、区切られた文脈が異なれば、それぞれのモデルを独立して実装し進化させることが可能です。とはいえ、区切られた文脈そのものが独立しているわけではありません。完全に独立したコンポーネントを集めるだけでは、システムは構築できません。システムとして機能するには、コンポーネントどうしの連係が必要です。区切られた文脈も同じです。個々の区切られた文脈は、独立して進化できますが、お互いに連係することも必要です。結果として、区切られた文脈の間には、必ず接合部分が存在します。この接合部分を区切られた文脈間の「契約」と呼びます。

契約が必要な理由は、異なる区切られた文脈どうしでは、モデルと言葉が異なるからです。契約とは、利害が異なる複数の当事者間の取り決めです。ですから、取り決めを明文化し、お互いの利害を調整することが必要です。また、定義によって、区切られた文脈はそれぞれ独自の同じ言葉を使います。連係する場合、どちらの文脈の同じ言葉を使えばよいでしょうか。連係を実現するためには、このような関心事を重要な設計課題として取り組む必要があります。

この章では、区切られた文脈どうしの関係を明確にし、うまく連係するためのドメイン駆動設計の考え方とやり方を学びます。どの連係方法を選択するか

は、区切られた文脈を担当する開発チーム間でどう協力するかに依存します。ドメイン駆動設計のさまざまな連係方法を、チーム間の協力関係の違いによって、緊密な協力、利用者と供給者、そして互いに独立に分類して説明します。

4.1　緊密な協力

緊密な協力は、異なる区切られた文脈を担当するチームどうしで、お互いの意図をうまく伝えあうことに関係します。

区切られた文脈を連係する時に、もっとも単純な状況は、一つのチームが複数の区切られた文脈を開発する場合です。あるいは、二つのチームに共通の目標があり、お互いに相手チームの成功が自分のチームの成功につながる場合も同じです。繰り返しますが、重要なのはチーム間の意図の伝達と協力の質です。

チームどうしが緊密に協力するための、ドメイン駆動設計の次の二つのやり方を見てみましょう。

- 良きパートナー
- モデルの共有

4.1.1　良きパートナー

良きパートナー（Partnership）の関係は、区切られた文脈どうしの連係を臨機応変に実現します。APIの変更をしたチームが他のチームにそのことを伝えると、他のチームは協力的にその変更に対応します。大騒ぎになったり利害関係が衝突したりしません（図4‐1）。

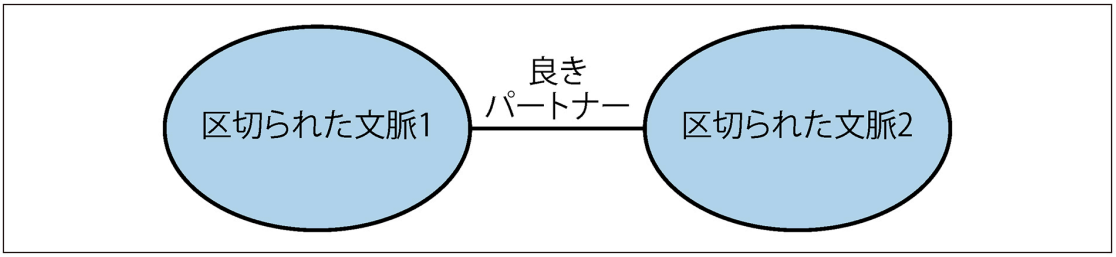

図4-1　良きパートナーの関係

この協力関係は双方向です。一方のチームが連係の約束事を決め、それを他のチームに押しつけることはしません。相違が見つかれば適切な方法で解決し

ます。そして連係に必要なさまざまな課題に対して二つのチームが協力して取り組みます。どちらのチームも相手チームの障害になることはありません。

良きパートナーの関係で連係するには、二つのチームが協力することに習熟し、目標の達成に強い意欲を持ち、頻繁に同期を行うことが必要です。技術的には、それぞれのチームが継続的インテグレーションを頻繁に実施することで、連係に関わる問題の発生を最小にします。

良きパートナーの関係は物理的に離れたチームどうしでは難しいかもしれません。同期や意図の伝達にいろいろな工夫が必要になるでしょう。

4.1.2　モデルの共有

区切られた文脈はモデルの境界です。しかし、場合によっては、複数の区切られた文脈で同じモデル（またはその一部）を実装することがあります。このやり方がモデルの共有（Shared Kernel）です。共有部分は、関係するすべての区切られた文脈の要求にもとづいて設計することが、きわめて重要です。さらに言えば、共有されたモデルは、それを使うすべての区切られた文脈と整合する必要があります。

たとえば、独自の認可モデルを全社システムで使うことを考えてみましょう。システム利用者はアクセス権限を付与されます。また所属部門に付与された権限を自分の権限として継承できます。そして、それぞれの区切られた文脈でこの認可モデルの変更が可能です。ある文脈で行った変更は、モデルを共有しているすべての文脈で、同じように適用される必要があります（図4‐2）。

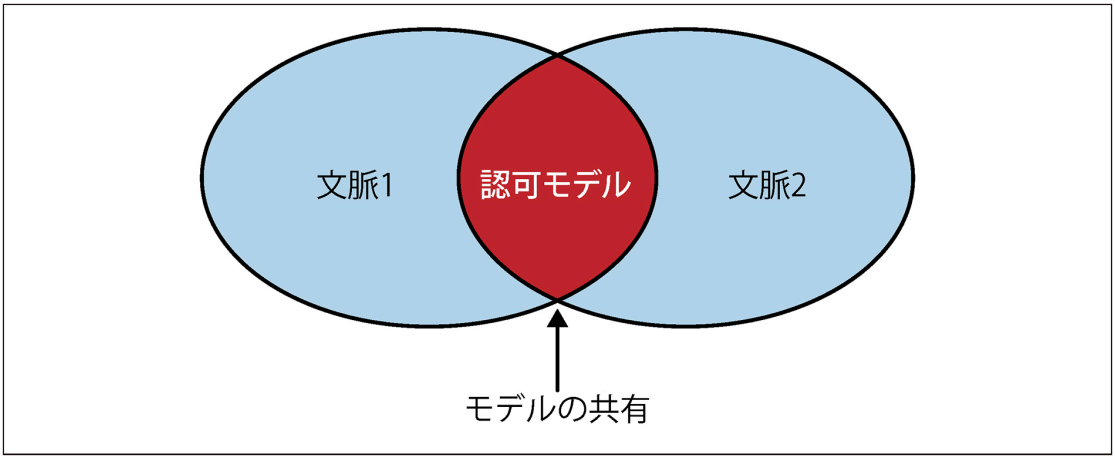

図4-2　モデルの共有

4.1.2.1　共有する範囲

認可モデルを共有していることで、区切られた文脈どうしが密結合になります。共有しているモデルへの変更は、すべての区切られた文脈にただちに影響します。ですから、変更の影響を小さくするために、モデルを共有する範囲は、どちらの文脈にも実装が必須になる部分に限定すべきです。理想的には、共有するモデルの対象は、連係するための契約と境界を越えて渡すデータ構造だけになるでしょう。

4.1.2.2　どう実現するか

モデルを共有する場合、共有部分のソースコードを変更した時は、それを利用しているすべての区切られた文脈で、その変更がただちに反映されるようにします。

単一リポジトリを採用している場合は、それぞれの区切られた文脈が同じソースコードを利用します。リポジトリが分かれている場合は、共有するモデルを独立したモジュールに切り出し、それぞれの区切られた文脈がこのモジュールを利用します。どちらのやり方でも、共有しているモデルを変更したら、そのモデルを共有するすべての区切られた文脈で、結合テストの自動実行が必要です。

共有部分に変更があれば、共有しているすべての区切られた文脈でその変更を取り込む必要があります。共有部分の変更を即時に反映しないとモデルの一貫性が失われます。つまり、それぞれの区切られた文脈が最新でない実装に依存し、データの不整合や実行時のエラーの原因になります。

4.1.2.3　モデルを共有するかどうかの判断

モデルを共有するかしないかは、コードの重複により発生するコストと共有するコストとの対比で判断します。モデルを共有すれば、区切られた文脈どうしが強く相互依存します。ですから、コードの重複により発生するコストよりもチーム間の調整にかかるコストが下回る時に限って、モデルを共有すべきです。別の言い方をすれば、共有するモデルに変更があった時に、区切られた文脈ごとに管理するソースコードで各チームが個別に対応するよりも、共有したソースコードへの変更を複数のチームで調整しながら進めるほうが負担が少ない場合に限り、モデルの共有を選択します。

継続的インテグレーションのコストと重複による変更コストのどちらが大きくなるかは、モデルの変わりやすさに依存します。モデルが頻繁に変われば、重複による変更コストのほうが大きくなるでしょう。したがって、モデルの共有を採用するのは変更が頻繁に起きる場所、つまり中核の業務領域となります。

ある意味で、モデルの共有は、前の章で説明した区切られた文脈の設計原則と矛盾します。関連する区切られた文脈を異なるチームで実装している時にモデルの共有を選択すると「一つの区切られた文脈は一つのチームが所有する」という原則と矛盾します。モデルの重なった部分、つまり共有されたモデルを、事実上、複数のチームで開発することになります。

モデルを共有するかどうかは慎重に判断してください。モデルの共有は、やむを得ず使うことがある例外的な連係方法です。たとえば、地理的に離れている、あるいはチーム間で組織の利害関係が衝突している場合です。このような状況では、良きパートナーのような、円滑な意図の伝達と協力体制は期待できません。密接に関連する機能の実装がうまく進まず、モデルの認識が合わず、そして、どちらがよいモデルかで意見が対立するでしょう。モデルの共有範囲をできるだけ小さくすることで変更の影響範囲を小さくし、変更のたびに結合テストを実行することで不整合を早期に発見するようします。

モデルの共有を選択する別の状況として、一時的な状況ですが、既存システムの段階的な置き換えがあります。この場合、コードベースの共有が、システムを段階的に区切られた文脈に分解していく時の、現実的な移行方法になる可能性があります。

最後になりますが、一つのチームが複数の区切られた文脈を所有して開発している場合、モデルの共有がよい選択になる可能性があります。この状況で、良きパートナーのように、必要に応じて同期させる臨機応変なやり方で開発を進めると、区切られた文脈の境界が不明確になってしまう可能性があります。区切られた文脈の境界を明確に定義するための手段として、モデルの共有を使うことが可能です。

4.2　利用者と供給者の関係

区切られた文脈どうしをつなぐ方法の2番目のグループは、利用者と供給者（Customer-Supplier）の関係です。図4‐3のように、片方の区切られた文脈が「供給者」となり、サービスを提供します。もう片方の区切られた文脈がサ

ービスの「利用者」となります。サービスを提供する側が「上流」で、サービスを利用する側が「下流」です。

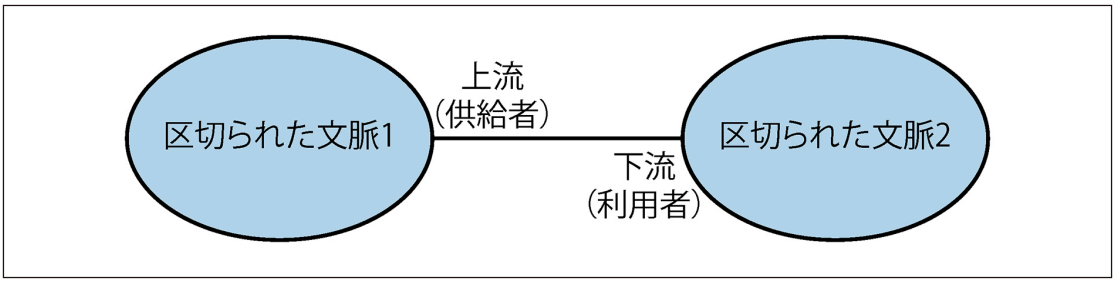

図4-3　利用者と供給者の関係

この関係は、緊密な協力とは異なり、自分たちのチームの成功にとって相手チームの成功は必須ではありません。その結果として、ほとんどの場合、二つのチームの間に力関係が生まれます。つまり、どう連係するかの契約をどちらかの一方のチームが決定します。

この節では、そういう力関係の違いによって生まれる、三つの連係方法を検討します。つまり、従属、モデル変換装置、そして共用サービスです。

4.2.1　従属する関係

上流チームが力を持ち、下流チームの都合を考慮しない状況があります。上流チームが自分たちのモデルに都合のよい連係の契約を決めます。このような力関係は外部サービスを利用する場合に発生します。あるいは組織の権力構造によって決まることがあります。

下流の開発チームが上流チームのモデルをそのまま受け入れられる場合は、二つの区切られた文脈の関係は従属（Conformist）になります。図4‐4のように、下流チームは上流チームのモデルに従います。

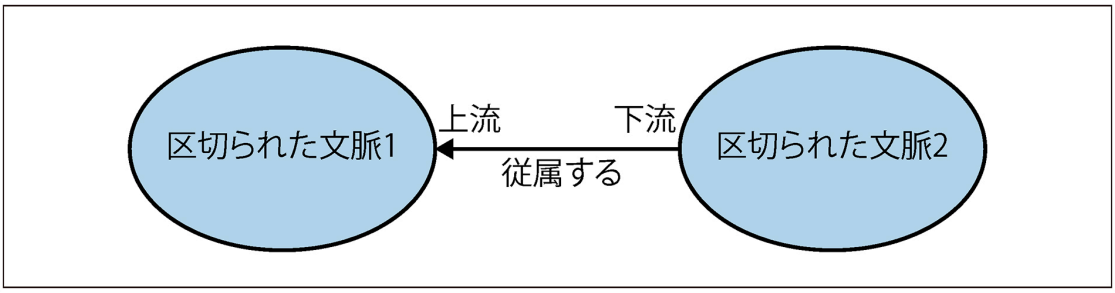

図4-4　従属する関係

下流チームは、モデルのある部分について、自分たちで決めることを放棄します。そうする理由はいくつかあります。たとえば、上流チームが提示する契約が業界標準であり安定したモデルの場合です。あるいは、上流チームが提供する内容が下流チームのニーズを十分に満たしている場合です。

では下流チームが上流チームが提示する契約を受け入れられない場合はどうすればよいでしょうか。そういう状況では次の方法を検討します。

4.2.2　モデル変換装置

このやり方は、従属する関係と同じように、上流のサービスが強い決定権を持ちます。しかし、下流の区切られた文脈のチームは上流のモデルをそのまま受け入れることはしません。そのかわり、上流の区切られた文脈のモデルを、自分たちのニーズに合わせて、変換する仕組みを用意します。この仕組みをモデル変換装置（AntiCorruption Layer）と呼びます（図4‐5）。

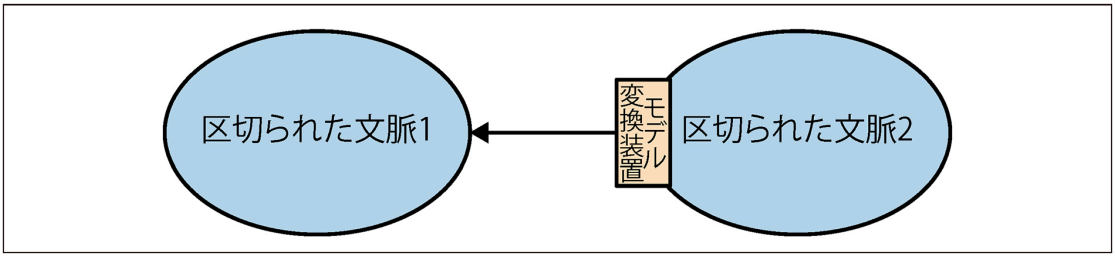

図4-5　モデル変換装置を使って連係する

モデル変換装置は、供給側のモデルに合わせることが好ましくない場合、あるいは合わせることに価値がない場合の解決方法です。たとえば次の場合です。

下流の区切られた文脈が中核の業務領域を含んでいる

中核の業務領域はきわめて重要です。モデルを作る時に細心の注意が必要です。供給側のモデルに引きずられると、中核の課題を表現するモデルをゆがめてしまう可能性があります。

上流のモデルが、利用する側にとって、非効率あるいは使いにくい

ごちゃごちゃしたモデルに合わせることは危険です。合わせた側のモデルもごちゃごちゃになってしまうでしょう。これは変更がやっかいで危険な既存シス

テムとの連係で起きがちな状況です。

供給側の契約が頻繁に変更される

サービスの利用者は、頻繁な変更から自分のモデルを守りたいと思うでしょう。モデル変換装置によって、供給側のモデルの変更が影響する範囲を限定できます。

モデルを作る観点からは、供給側のモデルを変換することで、下流のサービス利用者は、自身の区切られた文脈に関係しない外部の概念を取り除けます。その結果、サービスを利用する側の同じ言葉とモデルが単純でわかりやすくなります。

モデル変換装置のさまざまな実装方法は**第9章**で探求します。

4.2.3　共用サービス

共用サービス（Open-Host Service）はサービスを利用する側（下流）が決定権を持っている場合の連係方法です。サービスを提供する側（上流）は、サービスの利用者を保護し、利用者にできるだけよいサービスを提供しようとします。

サービス提供側（上流）のモデルに変更があっても、それが利用者（下流）に影響しないように、外部に提供する公開インターフェースを内部のモデルから切り離します。そうすることで外部に公開するモデルと内部の実装を別々のタイミングで変更できます（**図4‐6**）。

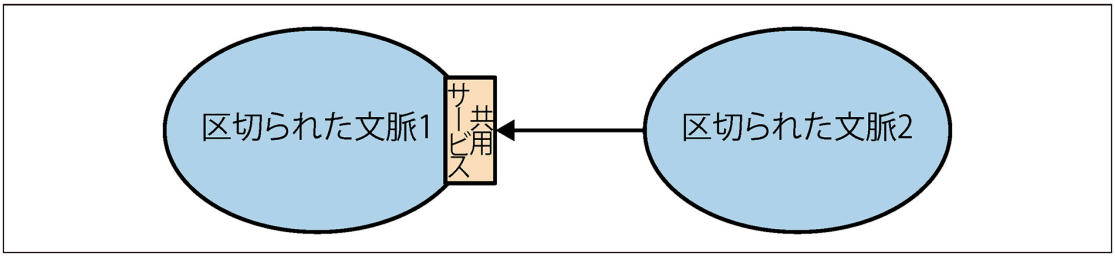

図4-6　共用サービスによる連係

サービス提供者が公開するインターフェースは、内部で使う同じ言葉に合わせる必要はありません。そうではなく、連係用途に特化した言葉を定義して、

利用する側が使いやすい通信規約（プロトコル）を提供します。この通信規約を公開された言葉（Published Language）と呼びます。

ある意味で、共用サービスはモデル変換装置の逆転です。つまり、モデルの変換を、利用する側ではなく、提供する側で行います。

区切られた文脈内部の実装モデルと外部と連係するモデルを切り離すことで、上流の区切られた文脈では、下流の文脈への影響を気にせずに、内部モデルの実装を変更できます。もちろん、自由に変更できるのは、内部モデルを変更しても下流の文脈がすでに使っている公開された言葉を維持できる場合だけです。

連係用のモデルを切り離すことには別の利点もあります。公開された言葉の異なるバージョンを並行して提供することで、サービスを利用する側は新しいバージョンに段階的に移行しやすくなります（図4‐7）。

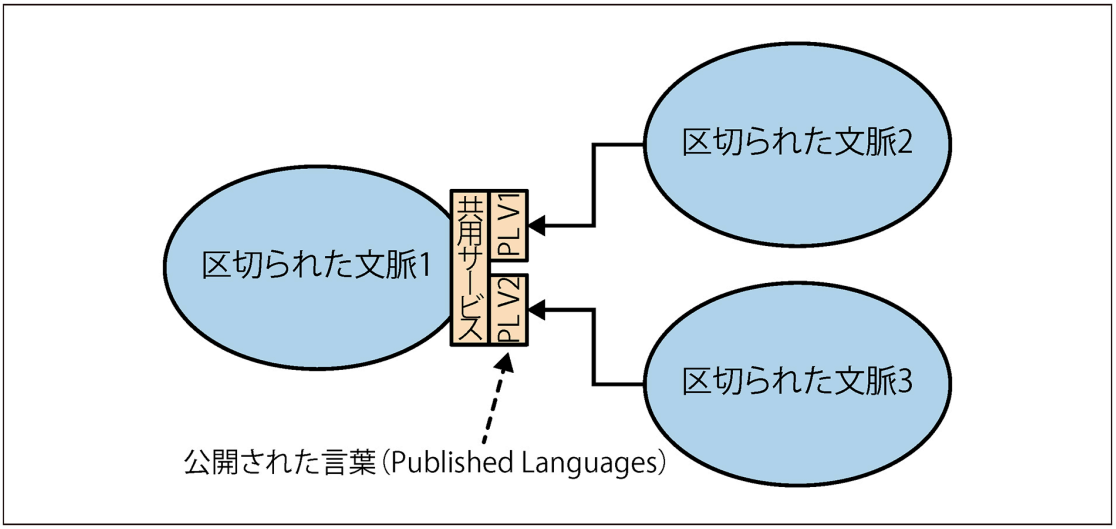

図4-7　複数バージョンの公開された言葉を提供する

4.3　互いに独立

最後の選択肢は連係を「しない」ことです。この方法を互いに独立（Separate Ways）と呼びます。互いに独立を選ぶ理由はいくつかあります。チームどうしが連係を希望しない場合、あるいは連係したくてもできない場合です。どういった状況でそうなるでしょうか。

4.3.1　意思疎通の問題

よくある理由は組織に起因するものです。組織が大きい場合、あるいは、部門間で利害が対立している場合です。チームどうしの協力や合意形成に苦労する状況だと、それぞれのチームが互いに独立して、複数の区切られた文脈で同じ機能を重複して開発したほうが、費用対効果は高いでしょう。

4.3.2　一般的な業務領域

業務が重複している場合も互いに独立を選択する理由になります。対象としている業務領域が、他社と同じでよい一般的な業務で、かつ、一般的な解決手段を簡単に組み込めるなら、それぞれの区切られた文脈ごとに、一般的な解決手段を取り入れるほうが費用対効果が高いでしょう。たとえばロギング用フレームワークの利用です。特定の区切られた文脈がロギング機能を共通サービスとして外部に提供することに、意味はないでしょう。それぞれの区切られた文脈が個別にロギング機能を組み込むほうが、共通のロギング機能を用意して複数の区切られた文脈で共同利用するより簡単です。こういう機能の開発は複数のチームで調整するよりも、重複して実装してしまうほうがよいでしょう。

4.3.3　モデルの違い

それぞれの区切られた文脈でモデルが違っているために、互いに独立を選択することもあるでしょう。たとえばモデルの違いが大きく、どちらかへの従属では対応できない場合です。あるいは必要な機能を文脈ごとに重複して作ってしまうほうが、モデル変換装置を作るよりコストが小さい場合です。このような場合、互いに独立を選ぶほうが費用対効果が高くなります。

> 中核の業務領域どうしをつなぐ場合は、互いに独立は避けるべきです。中核の業務領域を重複して実装することは、中核の業務領域はもっとも効率的かつ最適化された方法で実装する、という事業戦略にそむくことになります。

4.4　文脈の地図

ここまで区切られた文脈どうしを連係するさまざまな方法を分析してきました。システム全体の関係を文脈の地図（Context Map）として表現すると図4‐8になります。

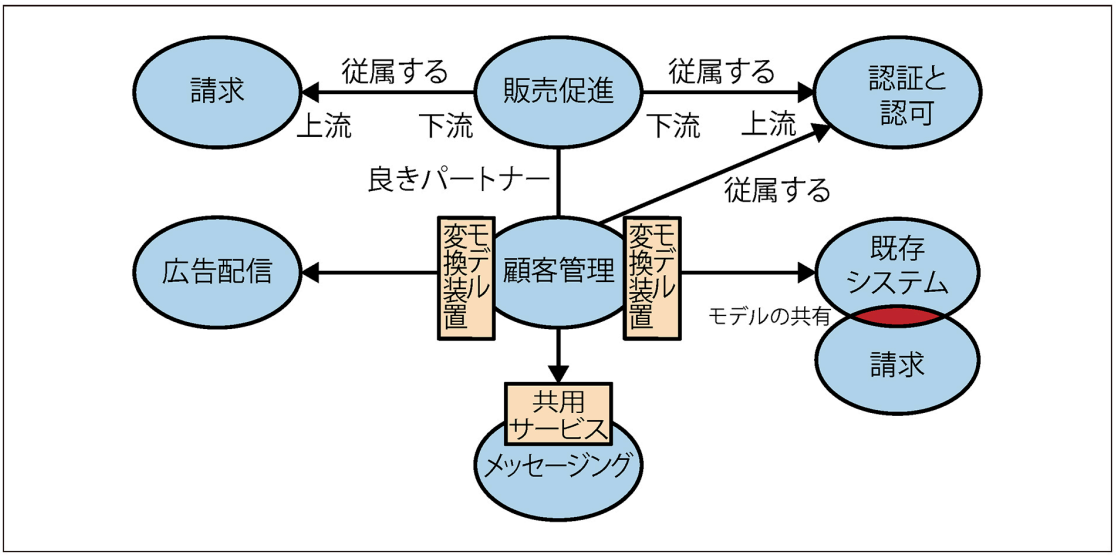

図4-8　文脈の地図

文脈の地図は、システム全体をどういう文脈に切り分け、それらをどう連係させるかを描いた絵です。区切られた文脈どうしの関係をこのように視覚化することで、さまざまなレベルで価値のある重要な洞察を得られます。つまり以下の内容です。

全体的な設計

文脈の地図は、システムのコンポーネントと、それぞれのコンポーネントが実装するモデルの集まりの全体像を提供します。

意図の伝わり方

文脈の地図はチーム間でどのように意図が伝わるかを表現します。たとえば、協力的な関係にあるのはどのチームか、あるいは、モデル変換装置や互いに独立のようにあまり親密ではないのはどのチームか、などを明らかにします。

組織上の課題

文脈の地図が組織上の課題を明らかにすることもあるでしょう。たとえば、ある上流チームと関係するすべての下流チームがモデル変換装置を開発している

としたら、それはどういう意味でしょうか。あるいは、あるチームが互いに独立の関係で囲まれているとしたら、それは何を意味するでしょうか。

4.4.1　最新の状態を維持する

理想的には、プロジェクトを開始する時に文脈の地図を作成し、区切られた文脈の追加や既存の区切られた文脈に変更があった場合、地図を描き直すべきです。

文脈の地図を作るには、多くのチームの開発内容を知ることが必要です。ですから文脈の地図を更新する活動は、すべてのチームが共同して進めるのがよいでしょう。つまり、それぞれのチームは、他の区切られた文脈とどう連係しているかの最新状態を地図に反映することに責任を持ちます。

文脈の地図はContext Mapper（**https://contextmapper.org**）などの道具を使えばコードとして管理できます。

4.4.2　限界

文脈の地図を描くのはたいへんな作業であることに注意してください。区切られた文脈が複数の業務領域を対象としている場合、複数の連係方法を同時に使うこともあります。たとえば**図4‐9**では、区切られた文脈を連係するために、二つの方法、つまり良きパートナーとモデル変換装置を使っています。

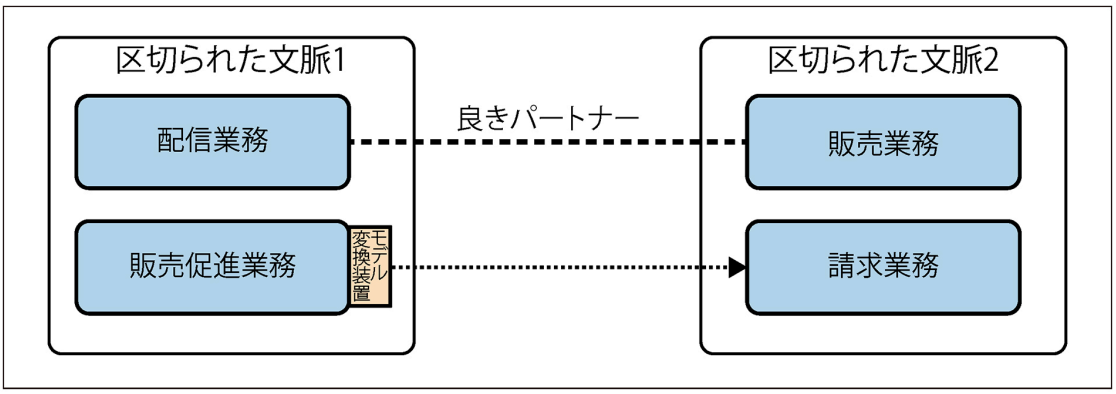

図4-9　入り組んだ文脈の地図

もっと言えば、区切られた文脈の対象となる業務領域が一つであっても連係のやり方が複数になることがあります。たとえば業務領域を構成するモジュールの間で異なる連係方法を必要とする場合です。

4.5　まとめ

区切られた文脈は完全に独立しているわけではありません。他の文脈と相互につながることが必要です。区切られた文脈どうしの連係方法は次のとおりです。

良きパートナー

区切られた文脈どうしを連係する時にそれぞれのチームが臨機応変に対応します。

モデルの共有

二つ以上の区切られた文脈を連係するために、必要最小限のモデルを共有します。

従属

サービスを利用する側が、サービスを提供する側のモデルに合わせます。

モデル変換装置

サービスを利用する側が、サービスを提供する側のモデルを自分のモデルに合うように変換します。

共用サービス

サービスを提供する側が、サービスを利用する側のニーズに最適化した公開された言葉を実装します。

互いに独立

連係のために協力する労力よりも、同じ機能を重複して実装したほうが負担が少ない場合の選択肢です。

区切られた文脈どうしの連係は文脈の地図で可視化できます。この地図は、システムの全体構造、チーム間の意図の伝わり方、そして組織上の課題を明らかにします。

ここまでは、事業活動を分析し理解するためのドメイン駆動設計の考え方とやり方を学んできました。次の**第II部**では、どう実装するかに観点を移して、業務ロジックの実装方法、アプリケーションの全体構造、そしてシステムのコンポーネント間の通信方法について学びます。

4.6　演習問題

1. 中核の業務領域に使ってはいけない連係方法はどれですか？
   A. モデルの共有
   B. 共用サービス
   C. モデル変換装置
   D. 互いに独立
2. モデル変換装置を使う可能性が高い下流の業務領域はどれでしょう？
   A. 中核の業務領域
   B. 補完的な業務領域
   C. 一般的な業務領域
   D. BとC
3. 共用サービスを提供する可能性が高い上流の業務領域はどれでしょう？
   A. 中核の業務領域
   B. 補完的な業務領域
   C. 一般的な業務領域
   D. AとB
4. ある意味で区切られた文脈の原則違反となるのはどの連係方法ですか？
   A. 良きパートナー
   B. モデルの共有
   C. 互いに独立
   D. どの連係方法も文脈を切り分ける原則には違反していない